

Pythonia

Manual de ruta de especialización

Automatización de oficina

Ruta de especialización

Que Python haga el trabajo repetitivo por ti.



6 módulos | 12 lecciones | 12 semanas en paralelo al tronco

Repasa, practica y registra cada ejecución en tu Cuadernillo-Diario.

Antes de empezar

Automatizar archivos, planillas, documentos, correos, sitios web y tareas programadas, con robots confiables que registran lo que hacen y se recuperan de los errores.

Esta ruta crece junto al tronco común. Cada módulo se desbloquea cuando completas sus lecciones requisito del tronco, así nunca llegas a la especialidad con la base floja. En la app, el módulo aparece en el nivel indicado del árbol; si ya cumples los requisitos antes, puedes adelantarlo.

Todo el código de este manual fue ejecutado: la salida que ves es la real. Cada lección incluye una **práctica evaluable** centrada en la lógica de la especialidad, sin librerías externas, para que puedas corregirla en la app. Cada módulo cierra con una **misión** y la ruta termina con un **proyecto final** (el jefe de la temporada).

Tu plan

Módulo	Aparece en	Requisitos del tronco
1. Archivos y carpetas Recorrer carpetas con pathlib Renombrar y mover en lote	Nivel 2	N2.4 Archivos, CSV y JSON
2. Excel con openpyxl Leer y escribir planillas Formato y consolidación	Nivel 2	N2.4 Archivos, CSV y JSON N2.1 Comprensiones
3. Word y PDF Documentos Word con python-docx PDF con ReportLab	Nivel 3	N2.6 Programación orientada a objetos I N3.3 Context managers
4. Correo automático Construir correos con adjuntos Envío seguro y combinación de correspondencia	Nivel 3	N2.5 Excepciones a fondo N3.8 Logging profesional
5. Automatización web Controlar el navegador con Playwright Extraer datos de páginas	Nivel 4	N3.6 Expresiones regulares N3.9 APIs y HTTP
6. Tareas programadas y robustez Programar ejecuciones Reintentos, registro y alertas	Nivel 4	N3.2 Decoradores N3.8 Logging profesional
Proyecto final: Robot de reportes semanales	Al cerrar	Todos los módulos

Ritmo sugerido: 30 a 45 minutos al día, 5 días por semana. Lo importante es no cortar la racha.

Módulo 1

Archivos y carpetas

Requisitos: N2.4 Archivos, CSV y JSON. Aparece en el Nivel 2 del árbol.

1 Recorrer carpetas con pathlib

Toda automatización empieza por encontrar archivos. `Path.iterdir()` lista una carpeta, `glob("**.pdf")` filtra por patrón y `rglob` busca también en subcarpetas.

Cada `Path` sabe su nombre (`.name`), su extensión (`.suffix`), si es archivo (`.is_file()`) y su tamaño y fechas (`.stat()`).

Agrupar archivos en un diccionario por extensión es el primer paso de cualquier organizador o auditoría de carpetas.

Ejemplo

```
1 from pathlib import Path
2
3 base = Path("descargas")
4 base.mkdir(exist_ok=True)
5 for nombre in ["foto.jpg", "informe.pdf", "datos.xlsx", "otra_foto.png",
6 ↪ "notas.txt", "acta.pdf"]:
7     (base / nombre).write_text("demo")
8
9 por_tipo = {}
10 for archivo in sorted(base.iterdir()):
11     if archivo.is_file():
12         por_tipo.setdefault(archivo.suffix, []).append(archivo.name)
13
14 for ext, nombres in sorted(por_tipo.items()):
15     print(f"{ext:6} {len(nombres)} -> {nombres}")
16 print("Total:", sum(len(v) for v in por_tipo.values()))
```

Salida real

```
.jpg 1 -> ['foto.jpg']
.pdf 2 -> ['acta.pdf', 'informe.pdf']
.png 1 -> ['otra_foto.png']
.txt 1 -> ['notas.txt']
.xlsx 1 -> ['datos.xlsx']
Total: 6
```

Consejos

- Ordena con `sorted()`: el orden de `iterdir()` depende del sistema operativo.
- `Path.home()` / "Downloads" apunta a tu carpeta real de descargas en cualquier sistema.

Ejercicio

Lista los 5 archivos más pesados de una carpeta y sus subcarpetas, mostrando el tamaño en MB.

Práctica evaluable**Agrupar archivos por extensión** · La lógica de la lección, sin librerías externas

El corazón de un organizador es decidir la extensión de cada archivo. Escribe

`agrupar_por_extension(nombres)`, que devuelve `{extensión: [nombres ordenados]}`. La extensión es lo que sigue al último punto, en minúsculas y con el punto (`".pdf"`). Los archivos sin extensión, o cuyo único punto está al inicio (`".bashrc"`), van a `"(sin extensión)"`.

Plantilla para empezar

```

1 def agrupar_por_extension(nombres):
2     """["b.PDF", "a.pdf", "notas"] -> {".pdf": ["a.pdf", "b.PDF"], "(sin
   ↳ extensión)": ["notas"]}"""
3     grupos = {}
4     for nombre in nombres:
5         pass
6     return grupos

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué método busca archivos también dentro de subcarpetas?

- a) `rglob`
- b) `iterdir`
- c) `walk_files`
- d) `glob`

2 Renombrar y mover en lote

`shutil.move` mueve archivos y `Path.rename` los renombra. Son operaciones que no se deshacen fácilmente, por eso todo robot serio tiene un modo simulación.

El patrón es simple: una bandera `simular=True` que solo muestra lo que haría. Revisas la salida y recién entonces ejecutas de verdad.

Normaliza siempre las extensiones con `.lower()`: `F0T0.JPG` y `foto.jpg` deben ir al mismo lugar.

Ejemplo

```

1  from pathlib import Path
2  import shutil
3
4  CATEGORIAS = {".jpg": "Imágenes", ".png": "Imágenes", ".pdf": "Documentos",
5  ↵ ".xlsx": "Planillas"}
6
7  def organizar(carpeta: Path, simular: bool = True) -> None:
8      for archivo in sorted(carpeta.iterdir()):
9          if not archivo.is_file():
10             continue
11             destino = carpeta / CATEGORIAS.get(archivo.suffix.lower(), "Otros")
12             accion = "MOVERÍA" if simular else "MOVIDO "
13             print(f"{accion}: {archivo.name} -> {destino.name}/")
14             if not simular:
15                 destino.mkdir(exist_ok=True)
16                 shutil.move(str(archivo), destino / archivo.name)
17
18  base = Path("descargas")
19  base.mkdir(exist_ok=True)
20  for n in ["foto.JPG", "informe.pdf", "notas.txt"]:
21      (base / n).write_text("demo")
22
23  organizar(base) # 1) simular
24  organizar(base, simular=False) # 2) ejecutar
25  print(sorted(p.name for p in base.iterdir()))

```

Salida real

```

MOVERÍA: foto.JPG -> Imágenes/
MOVERÍA: informe.pdf -> Documentos/
MOVERÍA: notas.txt -> Otros/
MOVIDO : foto.JPG -> Imágenes/
MOVIDO : informe.pdf -> Documentos/
MOVIDO : notas.txt -> Otros/
['Documentos', 'Imágenes', 'Otros']

```

Consejos

- Antes de mover, verifica si el destino ya existe y agrega un sufijo _1, _2 para no sobrescribir.
- Guarda un registro CSV de cada movimiento: te permite deshacer si algo sale mal.

Ejercicio

Agrega al organizador el manejo de nombres repetidos y un archivo historial.csv con origen, destino y fecha de cada movimiento.

Práctica evaluable**Nombres libres y plan de movimientos** · La lógica de la lección, sin librerías externas

Antes de mover archivos, el robot planifica sin tocar el disco.

- `nombre_libre(nombre, existentes)`: si el nombre no está en `existentes`, lo devuelve; si está, prueba `foto_1.jpg`, `foto_2.jpg`... (el número va antes de la extensión) hasta encontrar uno libre.
- `plan_de_movimientos(nombres, categorias)`: devuelve una lista de tuplas (`nombre`, `"Carpeta/nombre_final"`). La carpeta sale de categorías según la extensión en minúsculas (o "Otros"), y dos archivos iguales que van a la misma carpeta no deben pisarse.

Plantilla para empezar

```

1 def nombre_libre(nombre, existentes):
2     """nombre_libre("foto.jpg", {"foto.jpg"}) -> "foto_1.jpg" """
3
4
5 def plan_de_movimientos(nombres, categorias):
6     """[(nombre, "Carpeta/nombre_final"), ...]"""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Por qué un robot que mueve archivos debe tener modo simulación?

- Es más rápido
- Para ahorrar memoria
- Lo exige shutil
- Para revisar qué hará antes de ejecutar acciones difíciles de deshacer

Misión del módulo

Crea un organizador que ordene tu carpeta de descargas por tipo y fecha, con modo simulación.

Módulo 2

Excel con openpyxl

Requisitos: N2.4 Archivos, CSV y JSON, N2.1 Comprensiones. Aparece en el Nivel 2 del árbol.

3 Leer y escribir planillas

openpyxl lee y escribe archivos .xlsx sin necesidad de tener Excel instalado. Un libro (Workbook) tiene hojas, y cada hoja tiene celdas.

`ws.append([...])` agrega una fila al final; `iter_rows(values_only=True)` recorre filas como tuplas.

Puedes escribir fórmulas como texto ("`=B2*C2`"), pero openpyxl no las calcula: Excel lo hará al abrir el archivo.

Ejemplo

```
1 from openpyxl import Workbook, load_workbook
2
3 wb = Workbook()
4 ws = wb.active
5 ws.title = "Ventas"
6 ws.append(["Producto", "Cantidad", "Precio", "Subtotal"])
7 datos = [("Cable", 120, 4.5), ("Pintura", 30, 18.0), ("Rodillo", 45, 9.75)]
8 for fila, (prod, cant, precio) in enumerate(datos, start=2):
9     ws.append([prod, cant, precio, f"=B{fila}*C{fila}"])
10 ws.append(["TOTAL", None, None, f"=SUM(D2:D{len(datos) + 1})"])
11 wb.save("ventas.xlsx")
12
13 hoja = load_workbook("ventas.xlsx")["Ventas"]
14 for fila in hoja.iter_rows(min_row=2, values_only=True):
15     print(fila)
```

Salida real

```
('Cable', 120, 4.5, '=B2*C2')
('Pintura', 30, 18, '=B3*C3')
('Rodillo', 45, 9.75, '=B4*C4')
('TOTAL', None, None, '=SUM(D2:D4)')
```

Consejos

- ¿Necesitas el valor calculado? Calcúlalo en Python, o abre con `load_workbook(..., data_only=True)` un archivo que ya fue guardado por Excel.
- Para archivos .xls antiguos usa `xlrd` o conviértelos primero a .xlsx.

Ejercicio

Crea una planilla de inventario con 10 productos y una columna 'Estado' que diga 'Reponer' cuando el stock sea menor a 20.

Práctica evaluable

Filas con fórmulas y letras de columna · La lógica de la lección, sin librerías externas

- `letra_columna(n)`: convierte un número de columna en su letra de Excel: 1 → "A", 26 → "Z", 27 → "AA", 703 → "AAA".
- `filas_con_formulas(datos)`: recibe tuplas (producto, cantidad, precio) y devuelve las filas listas para `ws.append`, empezando en la fila 2: cada una con su fórmula de subtotal ("`=B2*C2`"), más una fila final [`"TOTAL", None, None, "=SUM(D2:D4)"`].

Plantilla para empezar

```
1 def letra_columna(n):
2     """1 -> "A", 27 -> "AA" """
3
4
5 def filas_con_formulas(datos):
6     """[("Cable", 120, 4.5)] -> [{"Cable", 120, 4.5, "=B2*C2"}, ["TOTAL", None,
    ↪ None, "=SUM(D2:D2)"] """
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

Si escribes "`=B2*C2`" con `openpyxl` y vuelves a leer la celda, obtienes...

- None
- Un error
- El resultado numérico
- El texto de la fórmula

4 Formato y consolidación

Una planilla automática debe verse profesional: encabezados con `Font` y `PatternFill`, formatos numéricos con `number_format`, anchos de columna ajustados y paneles inmovilizados.

Consolidar es unir datos de varias fuentes (sucursales, meses, archivos) en una sola tabla con una columna que indique el origen.

Calcula los totales en Python cuando necesites usarlos en el mismo robot: así no dependes de que alguien abra el archivo en Excel.

Ejemplo

```

1  from openpyxl import Workbook
2  from openpyxl.styles import Font, PatternFill, Alignment
3  from openpyxl.utils import get_column_letter
4
5  sucursales = {"Centro": [("Cable", 10, 3.2), ("Pintura", 4, 12.5)],
6                "Norte": [("Cable", 7, 3.2), ("Tornillos", 200, 0.15)]}
7
8  wb = Workbook()
9  ws = wb.active
10 ws.title = "Consolidado"
11 ws.append(["Sucursal", "Producto", "Cantidad", "Precio", "Subtotal"])
12 for suc, filas in sucursales.items():
13     for prod, cant, precio in filas:
14         ws.append([suc, prod, cant, precio, round(cant * precio, 2)])
15
16 for celda in ws[1]:
17     celda.font = Font(bold=True, color="FFFFFF")
18     celda.fill = PatternFill("solid", fgColor="0E3B43")
19     celda.alignment = Alignment(horizontal="center")
20 for col in ws.iter_cols(min_col=4, max_col=5, min_row=2):
21     for celda in col:
22         celda.number_format = "#,##0.00"
23 for i, col in enumerate(ws.columns, start=1):
24     ws.column_dimensions[get_column_letter(i)].width = max(len(str(c.value)) for c
25     ↪ in col) + 2
26 ws.freeze_panes = "A2"
27 wb.save("consolidado.xlsx")
28
29 total = sum(r[4] for r in ws.iter_rows(min_row=2, values_only=True))
30 print(f"Filas: {ws.max_row - 1} | Total: {total:,.2f}")
31 print("Anchos:", {k: v.width for k, v in ws.column_dimensions.items()})

```

Salida real

```

Filas: 4 | Total: 134.40
Anchos: {'A': 10.0, 'B': 11.0, 'C': 10.0, 'D': 8.0, 'E': 10.0}

```

Consejos

- Crea los estilos una sola vez y reutilízalos: cada objeto de estilo nuevo agranda el archivo.
- `ws.auto_filter.ref = ws.dimensions` agrega filtros a toda la tabla.

Ejercicio

Agrega una hoja 'Resumen' con el total por sucursal y una fila final en negrita con el total general.

Práctica evaluable

Anchos de columna y formato de miles · La lógica de la lección, sin librerías externas

- `anchos_columnas(filas)`: devuelve, para cada columna, el largo del texto más largo más 2 (None cuenta como texto vacío).
- `formato_es(numero)`: formatea con punto de miles y coma decimal, siempre con 2 decimales: 1234.5 → "1.234,50".

Plantilla para empezar

```

1 def anchos_columnas(filas):
2     """["Producto", 12], ["Cable", 1500.5]] -> [10, 8]"""
3
4
5 def formato_es(numero):
6     """1234.5 -> "1.234,50" """

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué propiedad deja fija la fila de encabezados al desplazarte?

- a) ws.fixed_rows
- b) ws.freeze_panes
- c) ws.header_lock
- d) ws.pin

Misión del módulo

Consolida las planillas de varias sucursales en un libro con formato profesional y una hoja de resumen.

Módulo 3

Word y PDF

Requisitos: N2.6 Programación orientada a objetos I, N3.3 Context managers. Aparece en el Nivel 3 del árbol.

5

Documentos Word con python-docx

python-docx crea y edita archivos .docx: títulos, párrafos con partes en negrita (runs), tablas con estilo, imágenes y saltos de página.

Un documento se arma como una secuencia: add_heading, add_paragraph, add_table. Cada párrafo puede tener varios runs con formato distinto.

Para informes repetitivos, conviene partir de una plantilla .docx con los estilos de tu organización y rellenarla con datos.

Ejemplo

```

1  from docx import Document
2
3  doc = Document()
4  doc.add_heading("Informe semanal de ventas", level=1)
5  p = doc.add_paragraph("Resumen generado automáticamente. ")
6  p.add_run("Total: 1.250,00").bold = True
7
8  tabla = doc.add_table(rows=1, cols=2)
9  tabla.style = "Light Grid Accent 1"
10  tabla.rows[0].cells[0].text, tabla.rows[0].cells[1].text = "Sucursal", "Ventas"
11  for suc, monto in [("Centro", "800,00"), ("Norte", "450,00")]:
12      fila = tabla.add_row().cells
13      fila[0].text, fila[1].text = suc, monto
14  doc.save("informe.docx")
15
16  leído = Document("informe.docx")
17  print([p.text for p in leído.paragraphs])
18  print(len(leído.tables[0].rows), "filas en la tabla")

```

Salida real

```

['Informe semanal de ventas', 'Resumen generado automáticamente. Total: 1.250,00']
3 filas en la tabla

```

Consejos

- Abre una plantilla con `Document("plantilla.docx")` para heredar logos, márgenes y estilos.
- Para reemplazar marcadores como `{{fecha}}` en una plantilla, recorre los runs de cada párrafo.

Ejercicio

Genera una carta personalizada por cada cliente de una lista, con su nombre, monto pendiente y fecha.

Práctica evaluable

Rellenar plantillas de documentos · La lógica de la lección, sin librerías externas

Para cartas e informes repetitivos se usan plantillas con marcadores. Escribe `rellenar(texto, datos)`, que reemplaza cada `{{clave}}` (con o sin espacios dentro de las llaves: `{{ nombre }}`) por `datos[clave]`, convertido a texto. Si falta un dato, lanza `KeyError("Falta el dato: clave")`.

Plantilla para empezar

```

1  import re
2
3
4  def rellenar(texto, datos):
5      """rellenar("Hola {{nombre}}", {"nombre": "Ana"}) -> "Hola Ana" """

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

En python-docx, ¿qué es un run?

- a) Una página
- b) Una tabla
- c) Un estilo
- d) Un tramo de texto con el mismo formato dentro de un párrafo

6 PDF con ReportLab

ReportLab genera PDF desde cero. El canvas dibuja en coordenadas (el origen está abajo a la izquierda) y es ideal para facturas, etiquetas y formularios.

Para documentos largos con párrafos que fluyen entre páginas se usa `platypus` (el mismo motor con el que se hicieron los manuales de Pythonia).

Para leer o verificar un PDF, `pypdf` extrae texto, cuenta páginas y une o separa archivos.

Ejemplo

```

1  from reportlab.lib.pagesizes import A4
2  from reportlab.pdfgen import canvas
3  from pypdf import PdfReader
4
5  items = [("Cable", 10, 3.2), ("Pintura", 4, 12.5)]
6  c = canvas.Canvas("factura.pdf", pagesize=A4)
7  ancho, alto = A4
8  c.setFont("Helvetica-Bold", 16)
9  c.drawString(50, alto - 60, "Factura 0001")
10 c.setFont("Helvetica", 11)
11 y = alto - 100
12 for prod, cant, precio in items:
13     c.drawString(50, y, f"{prod:<10} {cant:>4} x {precio:>6.2f} = {cant *
14     ↪ precio:>8.2f}")
15     y -= 18
16 total = sum(cant * precio for _, cant, precio in items)
17 c.drawString(50, y - 10, f"TOTAL: {total:.2f}")
18 c.save()
19 print(PdfReader("factura.pdf").pages[0].extract_text())

```

Salida real

```

Factura 0001
Cable      10 x   3.20 =   32.00
Pintura     4 x  12.50 =   50.00
TOTAL: 82.00

```

Consejos

- Mide antes de dibujar: `c.stringWidth(texto, fuente, tamaño)` te dice cuánto ocupa un texto.
- Registra una fuente TTF si necesitas símbolos que Helvetica no tiene.

Ejercicio

Genera una factura con logo, tabla de productos, impuesto y total, y numera automáticamente cada archivo.

Práctica evaluable

Líneas de factura y totales · La lógica de la lección, sin librerías externas

- `linea_factura(producto, cantidad, precio)`: una línea de ancho fijo: producto alineado a la izquierda en 12 caracteres, cantidad a la derecha en 5, " x ", precio a la derecha en 8 con 2 decimales, " = " y subtotal a la derecha en 10 con 2 decimales.
- `totales(items, iva=0.21)`: con tuplas (producto, cantidad, precio) devuelve {"subtotal": ..., "iva": ..., "total": ...} redondeados a 2 decimales.

Plantilla para empezar

```
1 def linea_factura(producto, cantidad, precio):
2     """Cable          10 x      3.20 =      32.00" """
3
4
5 def totales(items, iva=0.21):
6     """{"subtotal": ..., "iva": ..., "total": ...}"""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

En el canvas de ReportLab, el punto (0, 0) está...

- Arriba a la izquierda
- Abajo a la izquierda
- Arriba a la derecha
- En el centro

Misión del módulo

Genera un informe mensual en Word y su versión PDF a partir de la planilla consolidada.

Módulo 4

Correo automático

Requisitos: N2.5 Excepciones a fondo, N3.8 Logging profesional. Aparece en el Nivel 3 del árbol.

7 Construir correos con adjuntos

`email.message.EmailMessage` arma correos completos: remitente, destinatario, asunto, cuerpo en texto plano, versión HTML y adjuntos.

Un correo moderno es multiparte: el cliente de correo elige mostrar la versión HTML o la de texto plano.

Construir el mensaje y enviarlo son pasos separados. Así puedes revisar y probar los correos sin mandarlos.

Ejemplo

```

1  from email.message import EmailMessage
2  from pathlib import Path
3
4  Path("reporte.csv").write_text("sucursal,ventas\nCentro,800\n", encoding="utf-8")
5
6  msg = EmailMessage()
7  msg["From"] = "robot@miempresa.com"
8  msg["To"] = "gerencia@miempresa.com"
9  msg["Subject"] = "Reporte semanal"
10 msg.set_content("Hola,\nAdjunto el reporte semanal.\nSaludos.")
11 msg.add_alternative("<p>Hola,<br>Adjunto el <b>reporte semanal</b>.</p>",
12                    ↪ subtype="html")
13 msg.add_attachment(Path("reporte.csv").read_bytes(), maintype="text",
14                    ↪ subtype="csv", filename="reporte.csv")
15
16 print(msg["Subject"], "->", msg["To"])
17 print([parte.get_content_type() for parte in msg.walk()])
18 print("Adjuntos:", [a.get_filename() for a in msg.iter_attachments()])

```

Salida real

```

Reporte semanal -> gerencia@miempresa.com
['multipart/mixed', 'multipart/alternative', 'text/plain', 'text/html', 'text/csv']
Adjuntos: ['reporte.csv']

```

Consejos

- Usa `mimetypes.guess_type` para detectar el tipo de cualquier adjunto automáticamente.
- Guarda una copia del mensaje con `msg.as_bytes()` en un `.eml` para revisarlo con tu cliente de correo.

Ejercicio

Crea una función que reciba una lista de rutas y devuelva un `EmailMessage` con todos esos archivos adjuntos.

Práctica evaluable**Validar correos y armar cabeceras** · La lógica de la lección, sin librerías externas

- `correo_valido(c)`: True si tiene exactamente una @, algo antes, un dominio con al menos un punto que no esté en los extremos, y ningún espacio.
- `cabeceras(remite, destinatarios, asunto)`: devuelve {"From": ..., "To": "a@x.com, b@y.com", "Subject": ...}. Si algún correo es inválido, lanza `ValueError("Correos inválidos: ...")` con todos los inválidos separados por coma.

Plantilla para empezar

```

1 def correo_valido(c):
2     """correo_valido("ana@ejemplo.com") -> True"""
3
4
5 def cabeceras(remite, destinatarios, asunto):
6     """Diccionario de cabeceras del correo."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación¿Para qué sirve `add_alternative` con `subtype='html'`?

- Para enviarlo
- Para ofrecer una versión HTML del mismo cuerpo
- Para cifrar el correo
- Para adjuntar un archivo HTML

8**Envío seguro y combinación de correspondencia**

`smtplib.SMTP_SSL` envía por una conexión cifrada. Las credenciales nunca van en el código: se leen de variables de entorno.

Con Gmail u Outlook necesitas una contraseña de aplicación, no tu contraseña normal.

`string.Template` personaliza textos con marcadores `$nombre`: es la base de la combinación de correspondencia (un correo distinto para cada destinatario).

Ejemplo

```

1  import os, smtplib, ssl
2  from string import Template
3  from email.message import EmailMessage
4
5  PLANTILLA = Template("Hola $nombre, tu saldo pendiente es de $monto USD.")
6  clientes = [{"nombre": "Ana", "email": "ana@ejemplo.com", "monto": "120.00"},
7              {"nombre": "Luis", "email": "luis@ejemplo.com", "monto": "75.50"}]
8
9  def construir(cliente: dict) -> EmailMessage:
10     msg = EmailMessage()
11     msg["To"], msg["Subject"] = cliente["email"], "Recordatorio de pago"
12     msg.set_content(PLANTILLA.substitute(cliente))
13     return msg
14
15  def enviar(mensajes: list[EmailMessage]) -> None:
16     """Envía por SMTP cifrado. Credenciales SÓLO desde variables de entorno."""
17     usuario, clave = os.environ["SMTP_USER"], os.environ["SMTP_PASS"]
18     with smtplib.SMTP_SSL("smtp.gmail.com", 465,
19         ↪ context=ssl.create_default_context()) as s:
20         s.login(usuario, clave)
21         for m in mensajes:
22             m["From"] = usuario
23             s.send_message(m)
24
25  mensajes = [construir(c) for c in clientes]
26  for m in mensajes:
27     print(m["To"], "|", m.get_content().strip())
28  # enviar(mensajes) # descomenta cuando configures SMTP_USER y SMTP_PASS

```

Salida real

```

ana@ejemplo.com | Hola Ana, tu saldo pendiente es de 120.00 USD.
luis@ejemplo.com | Hola Luis, tu saldo pendiente es de 75.50 USD.

```

Consejos

- Haz una pausa breve entre envíos masivos para no ser marcado como spam.
- Primero envía todo a tu propia dirección como prueba.

Ejercicio

Lee los destinatarios desde una planilla Excel y registra en un CSV qué correos se enviaron y cuáles fallaron.

Práctica evaluable

Combinación de correspondencia · La lógica de la lección, sin librerías externas

Escribe `combinar(plantilla, clientes)` usando `string.Template`. Devuelve una tupla (`mensajes`, `errores`): `mensajes` es una lista de (`email`, `texto`) y `errores` la lista de emails cuyos datos no alcanzaron para completar la plantilla (esos clientes se saltean, sin detener al resto).

Plantilla para empezar

```

1 from string import Template
2
3
4 def combinar(plantilla, clientes):
5     """-> (mensajes, errores)"""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Dónde deben estar la usuaria y la contraseña SMTP?

- a) En el asunto del correo
- b) En el código, como constantes
- c) En un comentario
- d) En variables de entorno o un gestor de secretos

Misión del módulo

Envía a cada sucursal su reporte semanal como adjunto, con registro de envíos y reintento en fallos.

Módulo 5

Automatización web

Requisitos: N3.6 Expresiones regulares, N3.9 APIs y HTTP. Aparece en el Nivel 4 del árbol.

9

Controlar el navegador con Playwright

Playwright controla un navegador real: abre páginas, completa formularios, hace clic y lee resultados, igual que una persona pero sin cansarse.

Los selectores (#id, text=Entrar, roles) indican con qué elemento interactuar. Playwright espera solo a que el elemento esté listo.

Úsalo para portales sin API. Si existe una API oficial, préfiérela: es más rápida y estable.

Ejemplo

```

1  from pathlib import Path
2  from playwright.sync_api import sync_playwright
3
4  Path("form.html").write_text("""
5  <form><input id="usuario"><input id="clave" type="password">
6  <button type="button" onclick="msg.textContent = 'Bienvenido, ' +
7  ↵ usuario.value">Entrar</button></form>
8  <p id="msg"></p>""", encoding="utf-8")
9
10 with sync_playwright() as p:
11     navegador = p.chromium.launch(headless=True)
12     pagina = navegador.new_page()
13     pagina.goto(Path("form.html").resolve().as_uri())
14     pagina.fill("#usuario", "Leo")
15     pagina.fill("#clave", "secreto")
16     pagina.click("text=Entrar")
17     print(pagina.inner_text("#msg"))
18     navegador.close()

```

Salida real

Bienvenido, Leo

Consejos

- Instálalo con `pip install playwright` y luego `playwright install chromium`.
- `playwright codegen` URL graba tus clics y genera el código por ti.

Ejercicio

Automatiza el inicio de sesión en un sitio de práctica (como [the-internet.herokuapp.com](https://the-internet.herokuapp.com/login)) y toma una captura de pantalla.

Práctica evaluable

Esperas inteligentes · La lógica de la lección, sin librerías externas

Playwright espera solo a que un elemento esté listo: revisa una condición varias veces antes de rendirse.

Escribe `esperar_hasta(condicion, intentos=10, pausa=None)`: llama a `condicion()` hasta que devuelva algo verdadero y devuelve el número de intento en que lo logró (empezando en 1). Entre intentos fallidos, si se pasó una función pausa, la llama con el número de intento. Si nunca se cumple, lanza `TimeoutError("La condición no se cumplió tras 10 intentos")`.

Plantilla para empezar

```

1  def esperar_hasta(condicion, intentos=10, pausa=None):
2      """Devuelve el intento en que condicion() fue verdadera."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

Si un portal ofrece una API oficial, ¿qué conviene usar?

- a) Playwright siempre
- b) Copiar y pegar
- c) Selenium
- d) La API

10 Extraer datos de páginas

BeautifulSoup convierte HTML en un árbol que puedes recorrer con selectores CSS (`select`, `select_one`).

El flujo típico: obtener el HTML (requests o Playwright), extraer los datos, limpiarlos (números, fechas) y guardarlos en CSV o Excel.

Extrae con responsabilidad: respeta los términos de uso y el robots.txt del sitio, identifícate y no satures el servidor.

Ejemplo

```

1  from bs4 import BeautifulSoup
2  import csv, io
3
4  html = """<table id="precios"><tr><th>Producto</th><th>Precio</th></tr>
5  <tr><td>Cable</td><td>3,20</td></tr><tr><td>Pintura</td><td>12,50</td></tr></table>"""
6
7  sopa = BeautifulSoup(html, "html.parser")
8  filas = sopa.select("#precios tr")[1:]
9  datos = [{"producto": f.select("td")[0].text,
10           "precio": float(f.select("td")[1].text.replace(",", "."))} for f in filas]
11  print(datos)
12
13  salida = io.StringIO()
14  escritor = csv.DictWriter(salida, fieldnames=["producto", "precio"])
15  escritor.writeheader()
16  escritor.writerows(datos)
17  print(salida.getvalue().strip())

```

Salida real

```

[{'producto': 'Cable', 'precio': 3.2}, {'producto': 'Pintura', 'precio': 12.5}]
producto,precio
Cable,3.2
Pintura,12.5

```

Consejos

- Si los datos aparecen solo después de cargar JavaScript, obtén el HTML con Playwright (`pagina.content()`).
- `pandas.read_html` convierte tablas HTML en DataFrames en una línea.

Ejercicio

Extrae la tabla de un sitio de práctica, limpia los números y guárdala en Excel con formato.

Práctica evaluable

Limpiar datos extraídos de una página · La lógica de la lección, sin librerías externas

Lo que se extrae de una web llega como texto sucio.

- `limpiar_precio(texto): "$ 1.234,50" → 1234.5` (quita símbolos y espacios, el punto es de miles y la coma es decimal).
- `filas_a_dicts(encabezados, filas)`: lista de diccionarios con los encabezados limpios (sin espacios en los bordes y en minúsculas) como claves. Si una fila no tiene la misma cantidad de celdas que los encabezados, lanza `ValueError`.

Plantilla para empezar

```
1 def limpiar_precio(texto):
2     """$ 1.234,50" -> 1234.5"""
3
4
5 def filas_a_dicts(encabezados, filas):
6     """[" Producto", "Precio "], [{"Cable", 3.2}] -> [{"producto": "Cable",
   ↪ "precio": 3.2}]"""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué hace `sopa.select("#precios tr")`?

- Elimina filas
- Cuenta columnas
- Devuelve todas las filas dentro del elemento con id precios
- Crea una tabla

Misión del módulo

Automatiza la descarga de un reporte desde un portal web de prueba y conviértelo en planilla.

Módulo 6

Tareas programadas y robustez

Requisitos: N3.2 Decoradores, N3.8 Logging profesional. Aparece en el Nivel 4 del árbol.

11 Programar ejecuciones

Un robot vale cuando corre solo. En Linux y macOS se usa cron (0 8 * * 1-5 significa a las 8:00 de lunes a viernes); en Windows, el Programador de tareas.

Dentro de Python, librerías como APScheduler o schedule programan tareas en un proceso que queda corriendo.

Entender cómo se calcula la próxima ejecución te ayuda a evitar sorpresas con fines de semana, feriados y zonas horarias.

Ejemplo

```

1 from datetime import datetime, timedelta
2
3 DIAS = ["lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo"]
4
5 def proxima_ejecucion(ahora: datetime, hora: int, solo_habiles: bool = True) ->
  ↳ datetime:
6     candidato = ahora.replace(hour=hora, minute=0, second=0, microsecond=0)
7     if candidato <= ahora:
8         candidato += timedelta(days=1)
9         while solo_habiles and candidato.weekday() >= 5: # 5 = sábado, 6 = domingo
10             candidato += timedelta(days=1)
11     return candidato
12
13 momento = datetime(2026, 10, 2, 9, 30) # viernes 9:30
14 for _ in range(3):
15     momento = proxima_ejecucion(momento, 8)
16     print(DIAS[momento.weekday()], momento.strftime("%d/%m/%Y %H:%M"))

```

Salida real

```

lunes 05/10/2026 08:00
martes 06/10/2026 08:00
miércoles 07/10/2026 08:00

```

Consejos

- Usa zoneinfo.ZoneInfo("America/Caracas") o tu zona para fechas con zona horaria explícita.
- Guarda la fecha de la última ejecución exitosa: si el equipo estuvo apagado, el robot sabrá qué le falta.

Ejercicio

Agrega a proxima_ejecucion una lista de feriados que también se salte.

Práctica evaluable**Próxima ejecución con feriados** · La lógica de la lección, sin librerías externas

Escribe `proxima_ejecucion(ahora, hora, feriados=())`: devuelve el próximo `datetime` a la hora indicada (minutos y segundos en 0) que sea posterior a `ahora`, que caiga de lunes a viernes y cuya fecha no esté en `feriados` (un conjunto de objetos `date`).

Plantilla para empezar

```

1 from datetime import datetime, timedelta, date
2
3
4 def proxima_ejecucion(ahora, hora, feriados=()):
5     """Próximo día hábil a la hora indicada."""
6     candidato = ahora.replace(hour=hora, minute=0, second=0, microsecond=0)

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

En cron, ¿qué significa `0 8 * * 1-5`?

- a) A las 0:08 todos los días
- b) El día 8 de cada mes
- c) Cada 8 minutos
- d) A las 8:00 de lunes a viernes

12 Reintentos, registro y alertas

Las redes fallan y los servidores se caen. Un robot robusto reintentará con espera creciente (backoff exponencial) en lugar de rendirse al primer error.

Cada intento fallido queda registrado con logging, y si se agotan los intentos, el robot avisa (correo, Telegram) y deja el error a la vista.

Un decorador de reintentos se escribe una vez y protege cualquier paso del robot.

Ejemplo

```

1  import functools, logging, sys, time
2
3  logging.basicConfig(level=logging.INFO, stream=sys.stdout, format="%(levelname)-7s
↳ %(message)s")
4  log = logging.getLogger("robot")
5
6  def reintentar(veces=3, espera=0.01, factor=2):
7      def decorador(func):
8          @functools.wraps(func)
9          def envoltura(*args, **kwargs):
10             pausa = espera
11             for intento in range(1, veces + 1):
12                 try:
13                     return func(*args, **kwargs)
14                 except ConnectionError as e:
15                     log.warning("Intento %d/%d falló: %s", intento, veces, e)
16                     if intento == veces:
17                         log.error("Sin más intentos: avisando a soporte")
18                         raise
19                     time.sleep(pausa)
20                     pausa *= factor
21             return envoltura
22         return decorador
23
24  llamadas = {"n": 0}
25
26  @reintentar(veces=3)
27  def descargar_reporte():
28      llamadas["n"] += 1
29      if llamadas["n"] < 3:
30          raise ConnectionError("el servidor no responde")
31      return "reporte.xlsx descargado"
32
33  log.info(descargar_reporte())

```

Salida real

```

WARNING Intento 1/3 falló: el servidor no responde
WARNING Intento 2/3 falló: el servidor no responde
INFO      reporte.xlsx descargado

```

Consejos

- Reintenta solo errores temporales (red, tiempo de espera), nunca errores de datos.
- La librería `tenacity` ofrece reintentos configurables listos para producción.

Ejercicio

Haz que, al agotar los intentos, el robot envíe un correo de alerta con el traceback adjunto.

Práctica evaluable**Backoff y decisión de reintentar** · La lógica de la lección, sin librerías externas

- `espera(intento, base=1, factor=2, maximo=30)`: segundos a esperar antes del reintento número `intento` (empezando en 1): `base * factor ** (intento - 1)`, sin pasar de `maximo`.
- `plan_de_esperas(intentos, **opciones)`: lista con las esperas de cada intento.
- `debe_reintentar(error)`: True solo para errores temporales: `ConnectionError` y `TimeoutError` (incluidas sus subclases).

Plantilla para empezar

```

1 def espera(intento, base=1, factor=2, maximo=30):
2     """espera(3) -> 4"""
3
4
5 def plan_de_esperas(intentos, **opciones):
6     """plan_de_esperas(4) -> [1, 2, 4, 8]"""
7
8
9 def debe_reintentar(error):
10     """True para ConnectionError y TimeoutError."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué es el backoff exponencial?

- Duplicar la espera entre cada reintento
- Reiniciar el equipo
- Reintentar sin pausa
- Cancelar al primer error

Misión del módulo

Programa tu robot para que corra los días hábiles, reintente ante fallos y te avise si algo sale mal.

Proyecto integrador

Robot de reportes semanales

Un robot que cada lunes reúne las planillas de las sucursales, las consolida, genera un informe en PDF y lo envía por correo, sin intervención humana.

Paso a paso

1. Detecta las planillas nuevas en una carpeta con pathlib y modo simulación.
2. Consolídalas en un Excel con formato y hoja de resumen.
3. Genera un informe PDF con totales y la comparación con la semana anterior.
4. Envíalo por correo con adjuntos, credenciales en variables de entorno.
5. Programa la ejecución los lunes a las 8:00 con reintentos y log rotativo.
6. Extra: alerta por Telegram si alguna sucursal no envió su planilla.

Criterios de éxito

Funciona sin errores con entradas normales y extrañas, está dividido en funciones o clases con nombres claros, tiene docstrings y tests donde corresponde, y registraste el proceso en la Plantilla de proyecto del Cuadernillo.

Checklist de cierre

- ☐ Misión cumplida: Archivos y carpetas
- ☐ Misión cumplida: Excel con openpyxl
- ☐ Misión cumplida: Word y PDF
- ☐ Misión cumplida: Correo automático
- ☐ Misión cumplida: Automatización web
- ☐ Misión cumplida: Tareas programadas y robustez
- ☐ Terminé el proyecto "Robot de reportes semanales"

Solucionario de autoevaluación

- 1. Recorrer carpetas con pathlib:** a) rglob. rglob es un glob recursivo: recorre todo el árbol de carpetas.
- 2. Renombrar y mover en lote:** d) Para revisar qué hará antes de ejecutar acciones difíciles de deshacer. Mover o renombrar en lote es fácil de hacer y difícil de revertir: primero se mira, después se ejecuta.
- 3. Leer y escribir planillas:** d) El texto de la fórmula. openpyxl guarda la fórmula pero no tiene motor de cálculo.
- 4. Formato y consolidación:** b) ws.freeze_panes. freeze_panes = "A2" inmoviliza todo lo que está arriba de la fila 2.
- 5. Documentos Word con python-docx:** d) Un tramo de texto con el mismo formato dentro de un párrafo. Un párrafo se compone de runs; cada uno puede tener negrita, cursiva o color propios.
- 6. PDF con ReportLab:** b) Abajo a la izquierda. Por eso se dibuja usando alto - margen para empezar desde arriba.
- 7. Construir correos con adjuntos:** b) Para ofrecer una versión HTML del mismo cuerpo. El cliente de correo muestra la versión HTML si puede, o el texto plano si no.
- 8. Envío seguro y combinación de correspondencia:** d) En variables de entorno o un gestor de secretos. Un secreto en el código termina tarde o temprano en un repositorio o en manos equivocadas.
- 9. Controlar el navegador con Playwright:** d) La API. Las APIs son más rápidas y no se rompen cuando cambia el diseño de la página.
- 10. Extraer datos de páginas:** c) Devuelve todas las filas dentro del elemento con id precios. select usa selectores CSS y devuelve una lista de elementos.
- 11. Programar ejecuciones:** d) A las 8:00 de lunes a viernes. Campos: minuto, hora, día del mes, mes, día de la semana.
- 12. Reintentos, registro y alertas:** a) Duplicar la espera entre cada reintento. Esperar cada vez más da tiempo al servicio para recuperarse.

Pistas y soluciones de las prácticas

Usa las pistas de a una, y solo después de intentarlo. Hay más de una solución correcta: si la tuya pasa las pruebas, compárala con esta para aprender otra forma de pensarlo.

1. Recorrer carpetas con pathlib · Agrupar archivos por extensión

Pista 1: `"copia.tar.gz".rpartition(".")` da `("copia.tar", ".", "gz")`.

Pista 2: Si base queda vacío `("bashrc")`, no hay extensión.

```
1 def agrupar_por_extension(nombres):
2     """{extensión: [nombres ordenados]}"""
3     grupos = {}
4     for nombre in nombres:
5         base, punto, ext = nombre.rpartition(".")
6         clave = "." + ext.lower() if punto and base else "(sin extensión)"
7         grupos.setdefault(clave, []).append(nombre)
8     return {k: sorted(v) for k, v in grupos.items()}
```

Por qué funciona: `rpartition` separa por el último punto y deja claro si hubo punto y si había algo antes. En tu IDE, `Path(nombre).suffix` hace lo mismo, con las mismas reglas.

2. Renombrar y mover en lote · Nombres libres y plan de movimientos

Pista 1: Usa un `while` que suba el número mientras el nombre exista.

Pista 2: `usados.setdefault(carpetas, set())` lleva los nombres ocupados de cada carpeta.

```
1 def nombre_libre(nombre, existentes):
2     """Primer nombre que no está en existentes."""
3     if nombre not in existentes:
4         return nombre
5     base, punto, ext = nombre.rpartition(".")
6     if not punto or not base:
7         base, punto, ext = nombre, "", ""
8     n = 1
9     while f"{base}_{n}{punto}{ext}" in existentes:
10         n += 1
11     return f"{base}_{n}{punto}{ext}"
12
13
14 def plan_de_movimientos(nombres, categorias):
15     """[(nombre, "Carpeta/nombre_final"), ...]"""
16     usados = {}
17     plan = []
18     for nombre in nombres:
19         ext = "." + nombre.rpartition(".")[2].lower() if "." in nombre else ""
20         carpeta = categorias.get(ext, "Otros")
21         ocupados = usados.setdefault(carpeta, set())
22         final = nombre_libre(nombre, ocupados)
23         ocupados.add(final)
24         plan.append((nombre, f"{carpeta}/{final}"))
25     return plan
```

Por qué funciona: Separar el plan de la ejecución es el modo simulación de la lección llevado al extremo: el plan es una lista que se puede revisar, guardar en un CSV o probar con tests antes de mover un solo archivo.

3. Leer y escribir planillas · Filas con fórmulas y letras de columna

Pista 1: `divmod(n - 1, 26)` da la letra de la derecha y lo que queda por convertir.

Pista 2: `chr(ord("A") + 2)` es "C".

```

1 def letra_columna(n):
2     """1 -> "A", 27 -> "AA" """
3     letras = ""
4     while n > 0:
5         n, resto = divmod(n - 1, 26)
6         letras = chr(ord("A") + resto) + letras
7     return letras
8
9
10 def filas_con_formulas(datos):
11     """Filas con subtotal en fórmula y fila de total."""
12     filas = [[p, c, pr, f"B{i}*C{i}"] for i, (p, c, pr) in enumerate(datos,
13     ↪ start=2)]
14     filas.append(["TOTAL", None, None, f"=SUM(D2:D{len(datos) + 1})"])
15     return filas

```

Por qué funciona: Las columnas de Excel son un sistema en base 26 sin cero: por eso se resta 1 en cada paso. `openpyxl` trae `get_column_letter`, que hace exactamente esto.

4. Formato y consolidación · Anchos de columna y formato de miles

Pista 1: `zip(*filas)` entrega las columnas.

Pista 2: Para intercambiar dos caracteres usa uno temporal: `, → X, . → ,, X → .`

```

1 def anchos_columnas(filas):
2     """Largo máximo de cada columna + 2."""
3     return [max(len(" if v is None else str(v)) for v in col) + 2 for col in
4     ↪ zip(*filas)]
5
6 def formato_es(numero):
7     """1234.5 -> "1.234,50" """
8     return f"{numero:,.2f}".replace(",", "X").replace(".", ",").replace("X", ".")

```

Por qué funciona: Medir por columnas con `zip(*filas)` evita índices. El intercambio con un carácter temporal es el truco clásico para reemplazar dos cosas entre sí sin pisarlas.

5. Documentos Word con `python-docx` · Rellenar plantillas de documentos

Pista 1: `re.sub` acepta una función: recibe cada coincidencia y devuelve el reemplazo.

Pista 2: Las llaves son especiales en regex: escápalas con `\{`.

```

1 import re
2
3 MARCADOR = re.compile(r"\{\s*(\w+)\s*\}")
4
5
6 def rellenar(texto, datos):
7     """Reemplaza {{clave}} por datos[clave]."""
8     def valor(m):
9         clave = m.group(1)
10        if clave not in datos:
11            raise KeyError(f"Falta el dato: {clave}")
12        return str(datos[clave])
13    return MARCADOR.sub(valor, texto)

```

Por qué funciona: Pasar una función a `re.sub` permite decidir el reemplazo de cada marcador y fallar con un mensaje claro si falta un dato, en lugar de dejar `{{fecha}}` impreso en la carta.

6. PDF con ReportLab · Líneas de factura y totales

Pista 1: En una f-string, `:<12` alinea a la izquierda en 12 y `>8.2f` a la derecha con 2 decimales.

Pista 2: Recorta el producto con `[ :12]` para que no desborde.

```

1 def linea_factura(producto, cantidad, precio):
2     """Línea de factura de ancho fijo."""
3     return f"{producto[:12]:<12}{cantidad:>5} x {precio:>8.2f} = {cantidad *
4     ↪ precio:>10.2f}"
5
6 def totales(items, iva=0.21):
7     """{"subtotal": ..., "iva": ..., "total": ...}"""
8     subtotal = round(sum(c * p for _, c, p in items), 2)
9     impuesto = round(subtotal * iva, 2)
10    return {"subtotal": subtotal, "iva": impuesto, "total": round(subtotal +
11    ↪ impuesto, 2)}

```

Por qué funciona: Las f-strings con ancho y alineación arman columnas perfectas en texto plano: la misma técnica sirve para el PDF con ReportLab, el correo o un log legible.

7. Construir correos con adjuntos · Validar correos y armar cabeceras

Pista 1: `c.count("@") != 1` descarta los casos sin @ o con varias.

Pista 2: Junta todos los inválidos antes de lanzar el error.

```

1 def correo_valido(c):
2     """Validación básica de un correo."""
3     if " " in c or c.count("@") != 1:
4         return False
5     usuario, dominio = c.split("@")
6     return bool(usuario) and "." in dominio and not dominio.startswith(".") and not
    ↪ dominio.endswith(".")
7
8
9 def cabeceras(remitente, destinatarios, asunto):
10    """Diccionario de cabeceras del correo."""
11    invalidos = [c for c in [remitente, *destinatarios] if not correo_valido(c)]
12    if invalidos:
13        raise ValueError("Correos inválidos: " + ", ".join(invalidos))
14    return {"From": remitente, "To": ", ".join(destinatarios), "Subject": asunto}

```

Por qué funciona: Validar antes de enviar evita rebotes y correos a medias. Informar todos los inválidos de una vez ahorra corregir la lista de a uno.

8. Envío seguro y combinación de correspondencia · Combinación de correspondencia

Pista 1: Crea el Template una sola vez, antes del bucle.

Pista 2: substitute acepta el diccionario del cliente directamente.

```

1 from string import Template
2
3
4 def combinar(plantilla, clientes):
5     """-> (mensajes, errores)"""
6     molde = Template(plantilla)
7     mensajes, errores = [], []
8     for cliente in clientes:
9         try:
10            mensajes.append((cliente["email"], molde.substitute(cliente)))
11        except KeyError:
12            errores.append(cliente["email"])
13    return mensajes, errores

```

Por qué funciona: El try dentro del bucle aísla cada cliente: un dato faltante no frena el envío a los demás, y la lista de errores te dice exactamente a quién revisar.

9. Controlar el navegador con Playwright · Esperas inteligentes

Pista 1: Un for con range(1, intentos + 1) numera desde 1.

Pista 2: En tu IDE, la pausa sería time.sleep; aquí se inyecta para poder probarla.

```

1 def esperar_hasta(condicion, intentos=10, pausa=None):
2     """Devuelve el intento en que condicion() fue verdadera."""
3     for intento in range(1, intentos + 1):
4         if condicion():
5             return intento
6         if pausa and intento < intentos:
7             pausa(intento)
8     raise TimeoutError(f"La condición no se cumplió tras {intentos} intentos")

```

Por qué funciona: Recibir la pausa como parámetro hace la función testeable al instante, sin esperar segundos reales. Es la misma idea que usan Playwright y las librerías de reintentos por dentro.

10. Extraer datos de páginas · Limpiar datos extraídos de una página

Pista 1: `ch.isdigit()` filtra los dígitos.

Pista 2: Primero quita los puntos de miles y después cambia la coma por punto.

```

1 def limpiar_precio(texto):
2     """$ 1.234,50" -> 1234.5"""
3     limpio = "".join(ch for ch in texto if ch.isdigit() or ch in ",.")
4     return float(limpio.replace(".", "").replace(",", "."))
5
6
7 def filas_a_dicts(encabezados, filas):
8     """Filas -> diccionarios con encabezados limpios."""
9     claves = [e.strip().lower() for e in encabezados]
10    resultado = []
11    for fila in filas:
12        if len(fila) != len(claves):
13            raise ValueError(f"La fila {fila} tiene {len(fila)} celdas y se
14                ↪ esperaban {len(claves)}")
15        resultado.append(dict(zip(claves, fila)))
16    return resultado

```

Por qué funciona: Normalizar en un solo lugar (encabezados en minúsculas, precios como float) hace que el resto del robot trabaje con datos previsibles, vengan de la web, de un Excel o de un correo.

11. Programar ejecuciones · Próxima ejecución con feriados

Pista 1: `weekday()` devuelve 5 y 6 para sábado y domingo.

Pista 2: Une ambas condiciones en el mismo `while` con `or`.

```

1 from datetime import datetime, timedelta, date
2
3
4 def proxima_ejecucion(ahora, hora, feriados=()):
5     """Próximo día hábil a la hora indicada."""
6     candidato = ahora.replace(hour=hora, minute=0, second=0, microsecond=0)
7     if candidato <= ahora:
8         candidato += timedelta(days=1)
9     while candidato.weekday() >= 5 or candidato.date() in feriados:
10        candidato += timedelta(days=1)
11    return candidato

```

Por qué funciona: Un solo bucle avanza día por día mientras el candidato no sirva, sea por fin de semana o por feriado. Recibir los feriados como parámetro permite cargarlos de un archivo sin tocar la función.

12. Reintentos, registro y alertas · Backoff y decisión de reintentar

Pista 1: `**opciones` reenvía `base`, `factor` y `maximo` a `espera`.

Pista 2: `ConnectionResetError` es subclase de `ConnectionError`.

```
1 def espera(intento, base=1, factor=2, maximo=30):
2     """Segundos de espera antes del reintentto."""
3     return min(base * factor ** (intento - 1), maximo)
4
5
6 def plan_de_esperas(intentos, **opciones):
7     """Esperas de cada intento."""
8     return [espera(i, **opciones) for i in range(1, intentos + 1)]
9
10
11 def debe_reintentar(error):
12     """True para errores temporales."""
13     return isinstance(error, (ConnectionError, TimeoutError))
```

Por qué funciona: Calcular las esperas por separado del bucle de reintentos permite probarlas y ajustarlas sin esperar de verdad. Reintentar solo errores temporales evita insistir con datos que nunca van a funcionar.